APPENDIX

A. Counterfactual intervention protocol

In this subsection, we introduce the complete pipeline for constructing and validating the counterfactual-intervention and meaning-preserving replacement pools used in our evaluation.

Pipeline. Our pipeline consists of four steps.

1) First, we tokenize the retrieved chunks and identify candidate tokens that can be replaced. For each token, we record its containing sentence and use spaCy to identify its entity type.
2) Second, we prompt an LLM to generate several token-level counterfactual alternatives for each candidate. Each alternative must preserve the token's entity type and role in the sentence while keeping the sentence grammatical and coherent.
3) Next, we remove alternatives that contain multiple tokens, break the sentence grammar, or do not match the original entity type.
4) We classify the remaining alternatives as `COUNTERFACTUAL`, `PARAPHRASE`, or `INVALID`, and retain only those labeled `COUNTERFACTUAL`. The resulting replacement pool is fixed and shared by all methods.

We then generate meaning-preserving alternatives, such as synonyms or aliases, for each selected token that is not a number or date. These alternatives are generated using WordNet [80] and an LLM, and only those that preserve the original meaning are retained. The generation prompts and validation rules for the two pools are given below.

1) Counterfactual Generation and Validation: We construct the counterfactual pool for all candidate tokens before running any TCE method. For each target token, we mark its position with `[[double brackets]]` in the complete sentence and provide the LLM with its entity type. Prompt P1 asks the editor to generate several single-token alternatives that retain this contextual role but express a different factual value.

P1 Counterfactual Intervention: Generation

SYSTEM You generate meaningful factual counterfactual tokens. Preserve type and grammar, but change information. Never return synonyms. Output strict JSON only.

USER For every target, generate factual counterfactual replacements for the token enclosed in `[[double brackets]]`. A replacement must be one token, retain the target's contextual semantic category and grammatical role, and make the sentence express an incompatible factual alternative. It must not be a synonym, alias, paraphrase, spelling variant, inflection, generic placeholder, hypernym, hyponym, deletion, or random noise. For a token inside a multi-token named entity, preserve its name slot. Type hints may be wrong; infer the actual type from the marked sentence and complete entity. Return a compact strict JSON object mapping each short ID to its candidate list.

INPUT
```
{"id": "...", "token": "...",
"semantic_type_hint": "...",
"part_of_speech": "...", "fine_grained_tag":
"...", "complete_entity": "...",
"entity_token_position": "...",
"marked_sentence": "...", "forbidden": [...]
}
```

Each generated candidate passes two validation stages. First, deterministic checks reject empty or multi-token outputs, unchanged forms, malformed text, and candidates that violate the token's grammatical or entity slot. Second, Prompt P2 determines whether the remaining edit changes the factual value while preserving the original semantic role.

P2 Counterfactual Intervention: Validation

SYSTEM You perform conservative semantic-relation classification for token edits. Output strict JSON only.

USER Classify each one-token edit using only the marked sentence. Return `COUNTERFACTUAL` only if the candidate keeps the same fine-grained semantic slot and grammar while asserting a different, incompatible factual value. Return `PARAPHRASE` for synonyms, aliases, inflections, hypernyms, hyponyms, and broader or narrower restatements. Return `INVALID` for a different semantic role, malformed text, random noise, or an ungrammatical sentence. Return one compact strict JSON object mapping each ID to exactly one label.

INPUT
```
{"id": "...", "original": "...",
"candidate": "...", "semantic_type_hint": "...",
"part_of_speech": "...", "complete_entity":
"...", "entity_token_position": "...",
"marked_sentence": "..." }
```

We retain only candidates labeled `COUNTERFACTUAL`. The accepted alternatives are ordered deterministically and stored with their token positions in a shared pool. Consequently, whenever two methods select the same token, they apply the same counterfactual replacement.

2) Meaning-Preserving Replacement: Specifically, for each selected token, we construct a meaning-preserving replacement. We first retrieve its candidate synonyms from WordNet [80] and then provide the token, its sentence, and the corresponding metadata to the LLM. As shown in Prompt P3, the LLM generates synonyms or aliases that change the surface form while preserving the original factual meaning. Since numbers and dates generally have no valid synonymous replacements, we leave them unchanged.

P3 Meaning-Preserving Replacement: Generation

SYSTEM You generate contextual synonyms and aliases without changing factual meaning. Output strict JSON only.

USER For each token enclosed in `[[double brackets]]`, generate meaning-preserving replacements that fit the original sentence. Each output must be a synonym, alias, abbreviation expansion, or equivalent rendering of the marked token. It must preserve the original referent, fact, relation, polarity, and grammatical role. Do not change any entity, value, date, or number. WordNet suggestions may be used when they fit the sentence. If the marked token belongs to a multi-token name, replace only the marked position. Return an empty list when no valid replacement exists; otherwise, return a strict JSON object mapping each ID to its candidate list.

INPUT
```
{"id": "...", "token": "...",
"part_of_speech": "...", "semantic_type": "...",
"marked_sentence": "...", "wordnet_suggestions":
[...], "forbidden": [...] }
```

Next, we merge the candidates obtained from WordNet and Prompt P3 and remove duplicated, unchanged, or malformed outputs. For each remaining candidate, we replace the marked token and construct the corresponding revised sentence. Prompt P4 then compares the original and revised

sentences to determine whether the replacement preserves both factual meaning and grammatical correctness.

P4 Meaning-Preserving Replacement: Validation

SYSTEM You are a strict factual-equivalence auditor for contextual paraphrases. Output strict JSON only.

USER Compare the original and revised sentences for each record. Return `true` only when the candidate changes the surface wording while preserving the original referent, fact, relation, polarity, and grammatical role. Return `false` if the candidate changes the meaning, is broader or narrower than the original token, produces an ungrammatical sentence, or duplicates a neighboring word. Return a strict JSON object mapping each ID to `true` or `false`.

INPUT `{"id": "...", "original": "...", "candidate": "...", "part_of_speech": "...", "semantic_type": "...", "marked_sentence": "...", "revised_sentence": "..." }`

Finally, we retain only the candidates accepted by Prompt P4 and order them deterministically. We assign one candidate to each token; if none is accepted, we use a deterministic surface variation, such as changing the capitalization of a proper name.

Factual counterfactual

`Paris→Berlin`
The treaty was signed in **Paris**.

`1998→2005`
The company was founded in **1998**.

Meaning-preserving

`physician→doctor`
The **physician** examined the patient.

`automobile→car`
The **automobile** crossed the bridge.

Fig. 13. Token-level examples under the shared replacement contract: blue changes facts; green preserves meaning.

B. Limitations

We conduct an extensive evaluation of token-level counterfactual explanations from multiple angles. Nevertheless, the present study has several limitations.

(1) Model and task coverage. Our experiments cover eight question-answering datasets and four open-weight LLMs from three model families, with model sizes ranging from 4B to 12B. This scope does not establish that the same conclusions hold for substantially larger or closed-source models, other languages, non-QA generation tasks, or RAG systems over multimodal and structured sources. Although the cross-model and cross-dataset results are consistent, broader evaluation is required before generalizing the findings to these settings.

(2) Automatic replacement validation. The factual and meaning-preserving replacement pipelines use automatic filters rather than an oracle. An accepted candidate may subtly change its grammatical function, semantic type, referent, or contextual specificity; a valid candidate may also be rejected because of an imperfect type hint, a rare surface form, or local ambiguity. Figure 13 contrasts two intended factual counterfactuals with two meaning-preserving edits. These examples illustrate the replacement contract, but whether an arbitrary edit satisfies it still depends on the complete sentence and retrieved context. The answer-blind prompts, deterministic checks, semantic audits, and shared pools improve consistency across methods; they do not prove that every replacement is semantically perfect. An oracle guarantee would require human review of every edit. We therefore interpret the results under the adopted automatic replacement contract and expose the complete prompts and replacement-pool hashes for auditing.

(3) Access to model internals and surrogate fidelity. `ReFlow` constructs its contribution graph from Transformer attention messages and answer gradients. It therefore requires access to model internals and is not directly applicable to closed-source models that expose generated text but not their intermediate states or gradients. Moreover, message contributions and residual maximum flow are tractable surrogates for a nonlinear generation process. The final candidate sets are verified by rerunning the target LLM, but the theoretical reduction to a linear number of candidates applies to the flow-based search objective; it does not guarantee the globally smallest intervention under the unrestricted black-box response function.

(4) Computational and context scale. Contribution-graph construction requires one forward pass and one backward pass through the target model, followed by a linear number of candidate verifications in the worst case. We evaluate top-5 retrieved chunks, use 384-token chunking when expert boundaries are unavailable, and run on two NVIDIA A100 40GB GPUs. Longer contexts, deeper retrieval, and larger models increase memory and latency even though the residual-flow optimization itself is efficient. Scaling contribution tracing to very long-context and production RAG systems remains an important direction.

C. Detailed Evaluation-Metric Definitions

In this subsection, we provide the formal definitions of the two groups of flip-rate metrics introduced in Section VI-A. Let Q denote the evaluation queries, y_q the gold answer of query q, and T_q the token set returned by an evaluated method. We denote the responses before and after the counterfactual intervention by

$$\mathcal{R}_q^{\text{orig}} := \mathcal{R}_q(C), \quad \mathcal{R}_q^{\text{CF}} := \mathcal{R}_q(C[T_q \leftarrow \Psi(T_q)]) \tag{10}$$

respectively.

Flip-rate metrics. The Answer Flip Rate (Ans-FR) measures the percentage of queries whose responses change after the counterfactual intervention:

$$\text{Ans-FR} = \frac{1}{|Q|} \sum_{q \in Q} \mathbb{I}\big[\mathcal{R}_q^{\text{orig}} \not\equiv \mathcal{R}_q^{\text{CF}}\big]. \tag{11}$$

Ans-FR compares only the two generated responses and does not use y_q.

To measure whether the intervention changes a correct response into an incorrect one, let the metric $f \in \{\text{EM}, \text{Acc}\}$ return $f(\mathcal{R}, y_q) = 1$ if response $\mathcal{R}$ is correct and 0 otherwise. We first construct the corresponding originally correct query set

$$Q_f = \big\{q \in Q : f(\mathcal{R}_q^{\text{orig}}, y_q) = 1\big\} \tag{12}$$

and then compute

$$\text{FR}_f^{\text{CF}} = \frac{1}{|Q_f|} \sum_{q \in Q_f} \mathbb{I}\big[f(\mathcal{R}_q^{\text{CF}}, y_q) = 0\big]. \tag{13}$$

We report $\mathrm{FR}_{\mathrm{EM}}^{\mathrm{CF}}$ and $\mathrm{FR}_{\mathrm{Acc}}^{\mathrm{CF}}$ as EM-FR and Acc-FR, respectively. Thus, their denominators contain only queries answered correctly before the intervention.

Counterfactual flip-rate metrics. A response may also change after a meaning-preserving intervention. To measure this surface-form sensitivity, let $T_q^{\mathrm{MP}} \subseteq T_q$ contain the selected tokens that are neither numbers nor dates. We replace these tokens with their meaning-preserving alternatives, leave the remaining selected tokens unchanged, and obtain

$$\mathcal{R}_q^{\mathrm{MP}} := \mathcal{R}_q\big(C[T_q^{\mathrm{MP}} \leftarrow \Psi^{\mathrm{Syn}}(T_q^{\mathrm{MP}})]\big) \tag{14}$$

as the meaning-preserving response. Its answer and correctness-conditioned flip rates are

$$\text{Ans-FR}^{\mathrm{MP}} = \frac{1}{|Q|}\sum_{q\in Q}\mathbb{I}\big[\mathcal{R}_q^{\mathrm{orig}} \not\equiv \mathcal{R}_q^{\mathrm{MP}}\big] \tag{15}$$

and

$$\mathrm{FR}_f^{\mathrm{MP}} = \frac{1}{|Q_f|}\sum_{q\in Q_f}\mathbb{I}\big[f(\mathcal{R}_q^{\mathrm{MP}}, y_q) = 0\big], \quad f \in \{\mathrm{EM}, \mathrm{Acc}\}. \tag{16}$$

Following Section VI-A, we subtract each meaning-preserving flip rate from its counterfactual counterpart:

$$\begin{aligned} \text{Ans-CFR} &= \text{Ans-FR} - \text{Ans-FR}^{\mathrm{MP}}, \\ \text{EM-CFR} &= \mathrm{FR}_{\mathrm{EM}}^{\mathrm{CF}} - \mathrm{FR}_{\mathrm{EM}}^{\mathrm{MP}}, \\ \text{Acc-CFR} &= \mathrm{FR}_{\mathrm{Acc}}^{\mathrm{CF}} - \mathrm{FR}_{\mathrm{Acc}}^{\mathrm{MP}}. \end{aligned} \tag{17}$$

A larger CFR indicates that the observed response change is more attributable to the changed factual information than to surface-form sensitivity.

Finally, token-level F1 and normalized EM produce identical binary flip decisions for our short-answer outputs, so we report only EM. We omit Acc-FR and Acc-CFR for PQA because its three-way categorical labels do not use the answer-containment Acc criterion adopted for the other datasets. For the binary-choice QTZ dataset, canonical choice normalization makes EM and Acc identical; therefore, we omit the redundant Acc metrics.

This section provides additional experimental details and complete results for the experiments in Section VI. Unless stated otherwise, all experiments use the same query sets, retrieval outputs, counterfactual intervention pool, target LLM, and implementation settings described in Section VI-A.

D. Datasets

We convert every benchmark to the same query–corpus interface while preserving its official questions and gold answers. For each dataset, we sample 1,000 queries and retrieve from the associated corpus summarized in Table II. Specifically, the datasets include:

- **HotpotQA** [105] is a Wikipedia-based multi-hop QA benchmark. Its questions require combining evidence from multiple documents, and its supporting-fact annotations make it suitable for testing whether a token explanation captures distributed answer evidence.
- **TimeQA** [106] evaluates time-sensitive question answering. Questions contain explicit or implicit temporal constraints, so the correct answer depends on locating evidence associated with the requested date or period rather than retrieving a timeless fact.
- **FinQA** [107] requires numerical reasoning over financial reports. Evidence may occur in report text or tables; we expose the corresponding report content to retrieval while retaining the benchmark answers for evaluation.
- **MuSiQue** [108] constructs multi-hop questions by composing connected single-hop questions while reducing shortcut cues. It therefore tests whether `ReFlow` can identify evidence that participates in a longer compositional reasoning chain.
- **QuaRTz** [109] is a binary-choice benchmark for qualitative scientific relationships. Answering requires applying textual rules about how one quantity increases or decreases with another; we use canonical choice normalization when computing its flip metrics.
- **TriviaQA** [110] is a large-scale open-domain trivia benchmark paired with distantly supervised evidence documents. Its long retrieval contexts and lexical variation stress both evidence localization and compact intervention.
- **2WikiMultiHopQA** [111] is a Wikipedia-based multi-hop benchmark constructed to expose intermediate reasoning steps. Questions connect evidence across articles through bridge, inference, and comparison relations.
- **PopQA** [112] contains entity-centric, open-domain factual questions spanning popular and long-tail entities. It tests whether retrieved non-parametric evidence can identify answer support beyond facts already likely to be stored in model parameters.

E. Additional Experimental Results

In this subsection, we report the complete results for the experiments in Sections VI-C–VI-E. Each full result also repeats the datasets shown in the main text.

Exp.1. Effectiveness of contract-priced search. We report the complete eight-dataset comparison of contract-priced search. `One-shot` solves the fixed-price problem only at $\lambda = 1$, `Grid` tests 33 logarithmically spaced prices, and `ReFlow` enumerates all candidate sets induced by the complete price range. The three methods use the same 1,000 queries, contribution graphs, counterfactual intervention pool, and target LLM within each dataset.

Figure 14 supports two observations. *1) Exact enumeration achieves the highest Ans-FR on every dataset.* Its improvement over `Grid` ranges from 0.1 to 2.5 percentage points and is largest on QTZ. *2) The improvement does not require a larger intervention on most datasets.* `ReFlow` uses fewer tokens than `Grid` on six datasets; on QTZ and TrQA, the higher Ans-FR requires only 1.06 and 0.36 additional tokens, respectively. Thus, a single price may over-select tokens, while a fixed grid may miss useful candidate sets.

Exp.2. Number of tokens used by `ReFlow`. For each query, we report the size of the token set returned by `ReFlow`. If

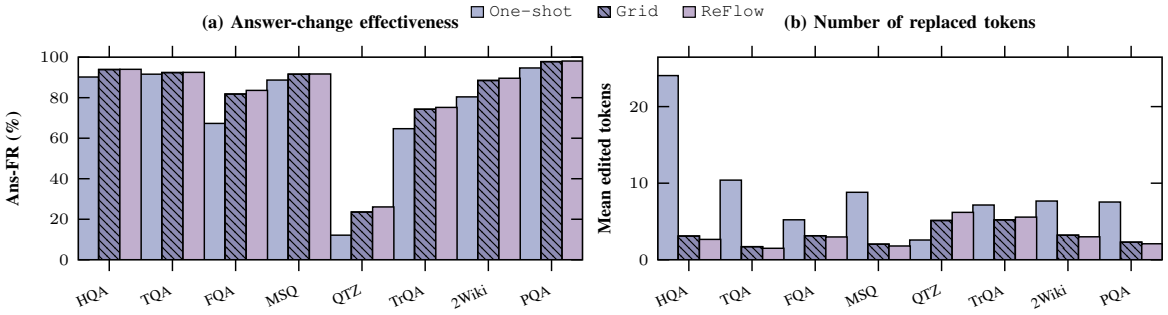
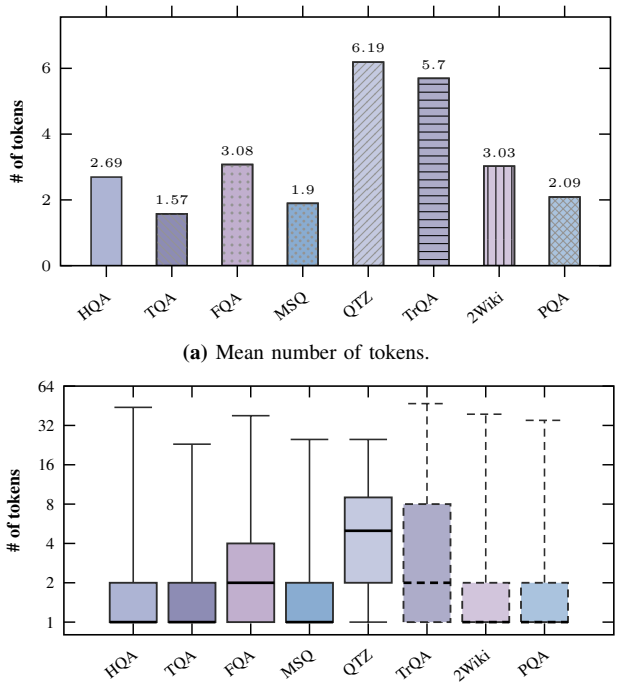


Fig. 14. Contract-priced-search variants on all eight datasets.



(a) Mean number of tokens.

(b) Per-query distribution.

Fig. 15. Numbers of tokens used by `ReFlow`.

no candidate changes the response, we use the largest set examined. Queries for which the original response cannot be evaluated are excluded. Figure 15 reports the dataset means and the complete per-query distributions.

Across the eight datasets, `ReFlow` uses 3.28 tokens per query on average. The dataset means range from 1.57 to 6.19 tokens, and the median is at most two tokens on seven datasets. Only a small fraction of queries require much larger candidate sets, producing the upper tails in Figure 15(b).

Exp.3. Effect of token budget. We extend Figure 2 to all eight datasets and three metrics. For each attribution baseline, we replace its top-ranked tokens under increasing budgets. For `ReFlow`, the star marks its result at the mean number of tokens it replaces, and the dashed line marks the same flip rate. The shaded regions and error bars are 95% Wilson intervals.

Figures 16– 18 show that `ReFlow` achieves a higher flip rate than the baselines at comparable token budgets. For example, `ReFlow` reaches 70.2–99.3% EM-FR with only 1.80–6.03 tokens on average. A baseline occasionally approaches or exceeds this flip rate only after replacing tens or hundreds of tokens. Thus, the advantage of `ReFlow` comes from selecting a more effective token set rather than using a larger intervention budget.

TABLE VII
AGGREGATE RETRIEVAL-DEPTH RESULTS ACROSS ALL EIGHT DATASETS.

k	Ans-FR	EM-FR	Acc-FR	Tok.	Cov.	Nodes	Edges	E2E (s)
1	75.2	70.5	66.3	2.19	96.9	99.5	309.3	2.66
3	81.2	71.6	65.7	2.93	98.8	303.5	509.0	7.01
5	82.2	71.8	66.4	3.26	97.0	455.5	506.3	11.94
10	82.0	69.0	63.0	3.57	99.6	1,042.4	614.4	24.50

Exp.4. Effect of retrieval depth k. We provide the complete results for the retrieval-depth experiment in Section VI-E. We vary $k \in \{1, 3, 5, 10\}$ while keeping the target LLM, contribution tracing, and all other settings unchanged. Figure 19 reports Ans-FR, EM-FR, Acc-FR, and their three control-adjusted counterparts on all eight datasets. For every setting, we rerun the LLM after applying meaning-preserving replacements to exactly the same selected token positions, which provides the paired controls required by the three CFR metrics. Table VII additionally reports the aggregate factual metrics and operational costs.

We make two observations. *1) Effectiveness is stable from $k = 3$ to $k = 10$.* At the default $k = 5$, aggregate Ans-FR, EM-FR, and Acc-FR are 82.2%, 71.8%, and 66.4%, and the corresponding CFRs are 58.1%, 57.9%, and 54.2%. No depth is best on every metric, and across $k \in \{3, 5, 10\}$ each of the six values varies by at most 3.5 points. On QTZ, the default uses the reader prompt of Table III, whereas the other depths use a choice-only prompt. *2) Increasing k raises the computational and intervention costs.* From $k = 1$ to $k = 10$, the mean intervention size increases from 2.19 to 3.57 tokens, the mean graph size from 99.5 to 1,042.4 nodes, and the end-to-end time from 2.66 to 24.50 seconds per query. These results support $k = 5$ as the default effectiveness–cost tradeoff.

Exp.5. Effect of receiver beam d. We next provide additional diagnostics for the receiver-beam experiment in Section VI-E. We vary $d \in \{16, 32, 48, 64\}$, where d is the maximum number of positive incoming contributions retained for each receiver token. Figure 20 provides one panel for each of the six metrics. Each CFR value uses a paired meaning-preserving intervention over the same token positions selected under the corresponding receiver beam. Table VIII reports the corresponding graph

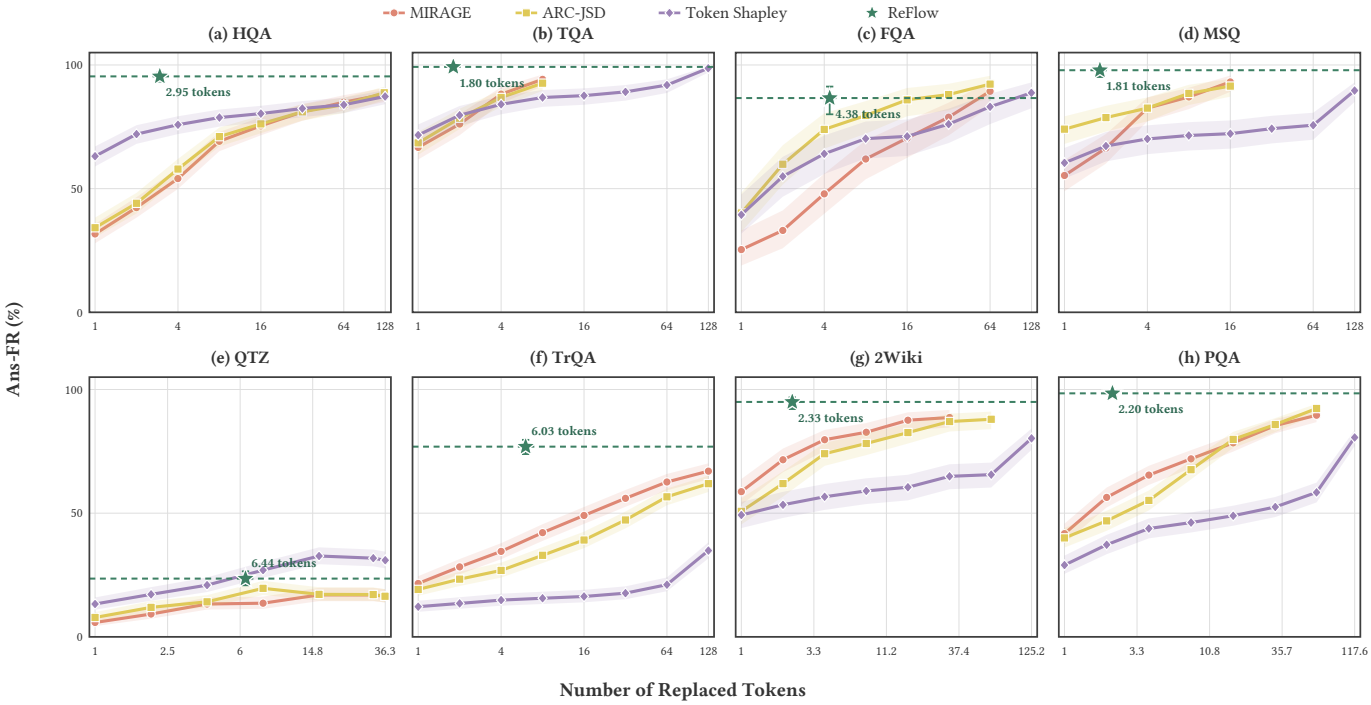


Fig. 16. Ans-FR under increasing token budgets; stars show `ReFlow` at its mean number of replaced tokens.

sizes.

We make two observations. *1) Increasing d improves FR and CFR, with diminishing returns.* From $d = 16$ to $d = 48$, aggregate Ans-FR, EM-FR, and Acc-FR increase by 21.7, 19.7, and 19.3 percentage points, while Ans-CFR, EM-CFR, and Acc-CFR increase by 20.2, 17.0, and 16.8 points. Increasing d further to 64 changes each of them by at most 1.9 points. *2) Larger beams substantially increase graph size.* From $d = 16$ to $d = 64$, the average number of active nodes and flow edges grows by $3.9\times$ and $9.9\times$, respectively. We therefore use $d = 48$ by default.

Exp.6. Effect of LLM backbone. We evaluate whether `ReFlow` depends on one model family by repeating the complete protocol with Qwen2.5-7B, Llama-3.1-8B, Mistral-Nemo-12B, and Qwen3-4B. All models use the same queries, retrieved chunks, and counterfactual intervention protocol.

Table IX reports all six metrics for every backbone–dataset pair. `ReFlow` remains effective across the four backbones: on the seven open-ended datasets, the lowest Ans-FR and Ans-CFR are 48.9% and 47.8%, respectively. QTZ is consistently harder because its binary choice protocol makes response changes less frequent. No backbone is uniformly best across the 48 dataset–metric combinations, showing that the result is not specific to the default Qwen2.5 model.

Exp.7. Complete RAG provenance results. We report the provenance experiment on all eight datasets. `Full` prompts the LLM with all top-5 retrieved chunks. `ReFlow`, `MIRAGE`, `ARC-JSD`, and `TS` instead use only the sentences containing their selected tokens, with all three baselines receiving the same per-query number of sentences as `ReFlow`.

Figure 21 shows that, after adding `TS`, `ReFlow` remains the strongest attribution method on all eight datasets under both F1 and EM. On QTZ, answers are compared by selected option, so F1 and EM coincide. Its gain over the strongest baseline ranges from 0.8 to 9.8 F1 points and from 0.8 to 9.5 EM points. On TrQA, for example, `ReFlow` obtains 71.5 F1, close to the 74.5 of `Full`; `MIRAGE`, `ARC-JSD`, and `TS` obtain 61.7, 44.6, and 50.0 F1, respectively. On PQA, all five methods are evaluated on the same 992-query population: `ReFlow` obtains 49.3 F1 and 39.7 EM, compared with 45.1/36.3 for `MIRAGE`, 35.6/28.9 for `ARC-JSD`, and 38.8/30.2 for `TS`. Thus, `ReFlow`'s selected sentences consistently preserve more of the information needed to reproduce the full-context response than the attribution baselines.

Exp.8. Contribution-graph size. Finally, we report the average contribution-graph size for each of the eight 1,000-query workloads.

Table X shows that the contribution graphs remain sparse: they contain 73–1,297 nodes and 188–964 edges, with average out-degree between 0.23 and 3.16. TrQA has the most nodes, whereas TQA has the largest average out-degree. This sparsity is consistent with the running-time results in Figure 23, where candidate enumeration adds little overhead beyond contribution-graph construction.

Exp.9. Candidate-enumeration efficiency. We compare the number of candidate sets actually needed in practice with the theoretical $n + 1$ bound, which ranges from 36.0 to 577.0 across datasets. In practice, only 8.2–27.3 candidate sets are needed, yielding a 4.5–$48.5\times$ reduction.

Exp.10. Running-time breakdown. `ReFlow` consists of two main computational stages: contribution-graph construction and exact candidate enumeration. The first stage performs forward and backward computation through the Transformer, whose cost grows quadratically with the number of prompt

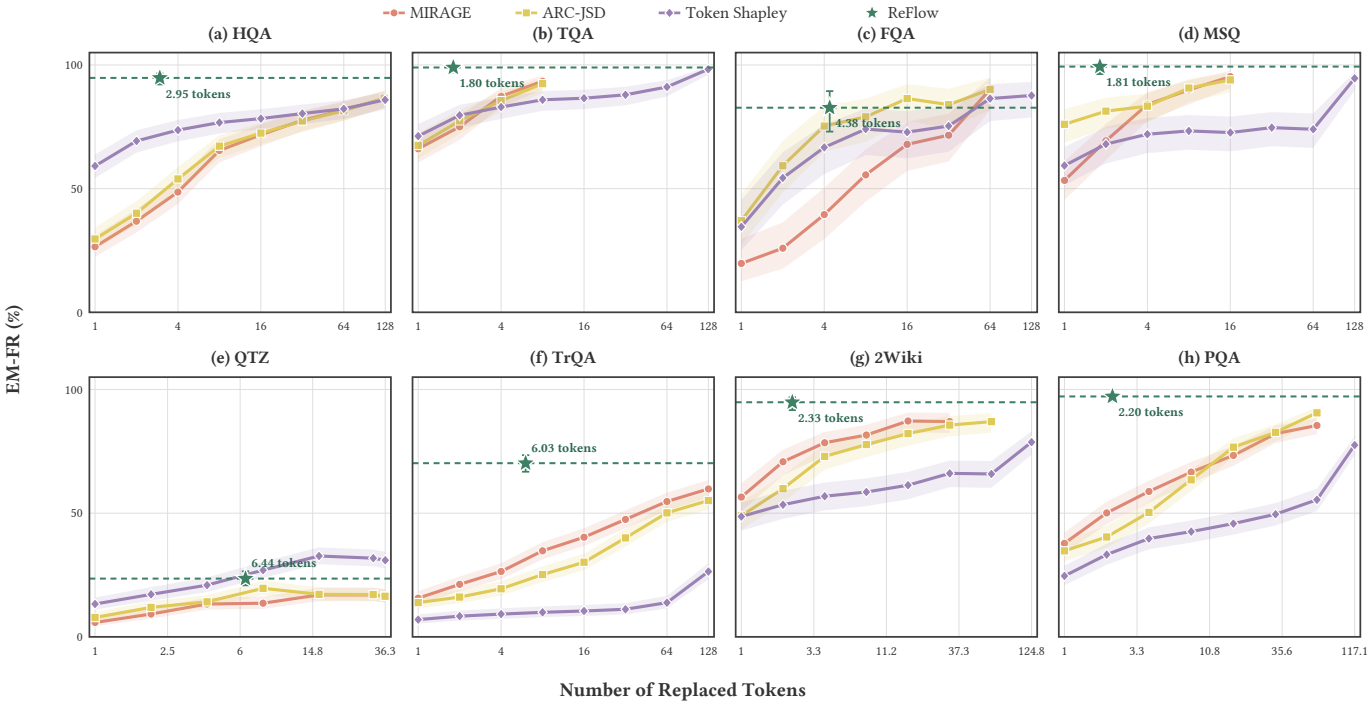


Fig. 17. EM-FR under increasing token budgets; stars show ReFlow at its mean number of replaced tokens.

TABLE VIII
GRAPH SIZE UNDER EACH RECEIVER BEAM. COLUMNS n AND m REPORT THE MEAN NUMBER OF NODES AND EDGES, RESPECTIVELY.

	$d = 16$		$d = 32$		$d = 48$		$d = 64$	
Dataset	n	m	n	m	n	m	n	m
HQA	1,236.5	3,160.8	2,445.4	9,524.9	3,639.2	18,889.0	4,819.2	31,197.6
TQA	1,242.4	3,166.5	2,465.2	9,616.2	3,671.4	19,078.0	4,861.5	31,494.2
FQA	1,217.5	3,119.2	2,375.2	9,312.1	3,574.6	18,615.0	4,781.9	31,051.9
MSQ	1,241.7	3,177.5	2,454.2	9,574.8	3,652.7	18,977.1	4,833.1	31,346.2
QTZ	1,241.2	3,166.4	2,485.6	9,664.0	3,652.1	18,858.6	4,885.2	31,533.1
TrQA	1,239.0	3,164.1	2,450.6	9,583.1	3,644.1	18,994.1	4,830.8	31,410.4
2Wiki	1,240.5	3,184.8	2,442.8	9,543.6	3,624.2	18,868.4	4,796.9	31,165.6
PQA	1,235.5	3,141.0	2,445.7	9,483.2	3,622.9	18,725.4	4,769.1	30,865.6

tokens and also depends on the model dimension and number of layers. The second stage performs maximum-flow computations on the constructed contribution graph, whose practical cost mainly depends on the number of graph nodes and edges. Since the contribution graph typically remains very sparse, candidate enumeration is much cheaper. As shown in Figure 23, graph construction takes 2.07 to 11.60 seconds per query, whereas exact enumeration requires only 0.008 to 0.201 seconds. On HQA, for example, graph construction takes 4.40 seconds, while all 46.4 maximum-flow computations together take only 0.137 seconds. Thus, graph construction dominates the running time, while exact candidate enumeration adds little overhead.

F. Further Analyses

This subsection reports the complete results of the analyses summarized in Sections IV-D and VI-C. All of them use Qwen2.5-7B-Instruct, the default contribution graphs, and the shared counterfactual pool, unless stated otherwise.

Flow versus graph scores. Table XI gives Ans-FR, Ans-CFR, and EM-FR of Graph-Score, Single-Flow, and ReFlow on all eight datasets. On QTZ, the variants were evaluated without the meaning-preserving control, so its CFR is not available.

Running time and memory. Table XII compares the per-query time and peak GPU memory of ReFlow and the baselines on 100 HQA queries on the same A100 GPU, including model loading. The lower part gives the number of LLM verification calls of ReFlow per query on the complete 1,000-query runs. The shared pool is built offline once per dataset; with Llama-3.1-8B-Instruct as the editor on one GPU it took 2.1, 1.7, and 0.4 hours for HQA, MSQ, and FQA, i.e., 7.5, 6.0, and 1.5 seconds per query. Contribution tracing does not change the model output: on 1,000 HQA queries, the traced and plain forward passes give identical next-token logits at all 9,467 response positions and identical greedy responses.

Accuracy of the first-order contribution. Table XIII reports, for 100 queries per dataset, the mean per-query Spearman correlation between $-r_q^{(\ell)}(j,i)$ and the exact change ΔS_q when the message is removed, the fraction of messages whose sign is predicted correctly, and the median relative error of the magnitude. *Top* uses the 200 messages with the largest

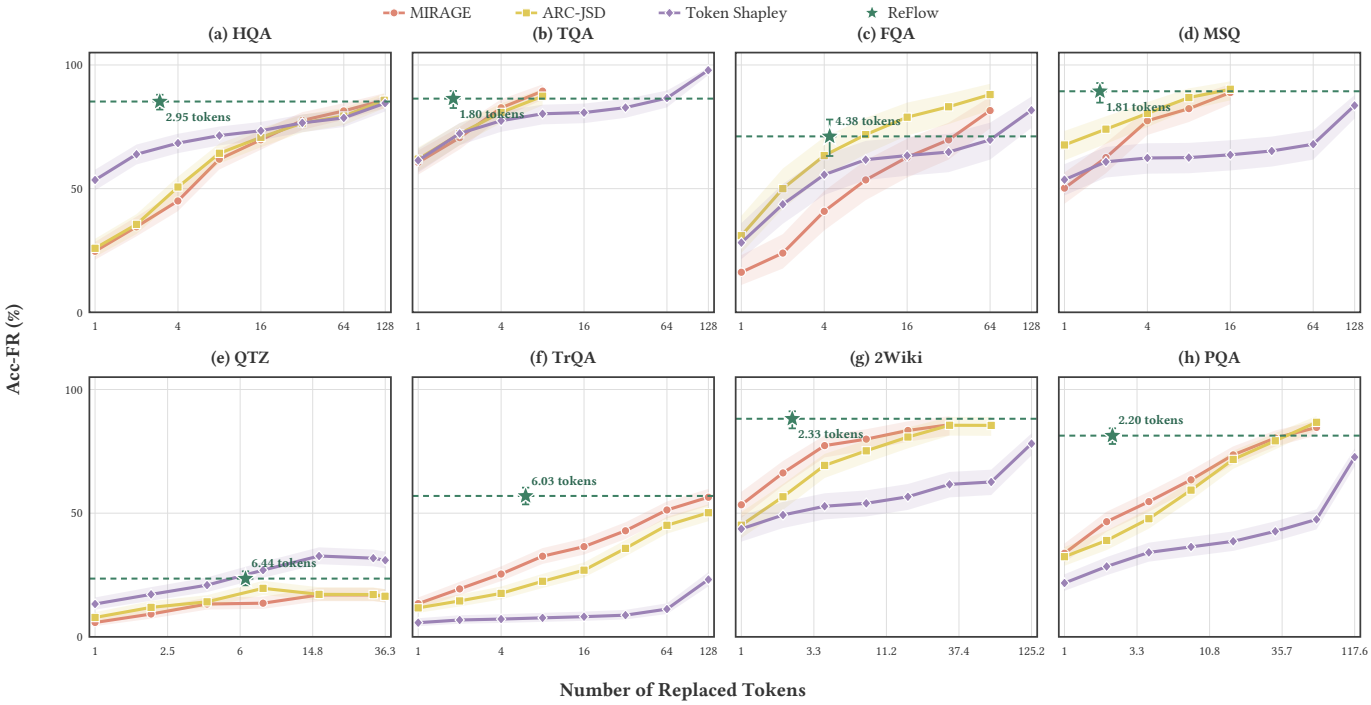


Fig. 18. Acc-FR under increasing token budgets; stars show `ReFlow` at its mean number of replaced tokens.

TABLE IX
DETAILED CROSS-MODEL RESULTS (%). SAND MARKS THE BEST BACKBONE IN EACH ROW.

Metric	Qwen2.5-7B	Llama-3.1-8B	Mistral-12B	Qwen3-4B
HQA				
Ans-FR	95.1	91.9	88.4	75.7
EM-FR	94.8	89.1	85.2	70.4
Acc-FR	85.2	79.4	78.8	62.5
Ans-CFR	56.5	76.1	76.2	64.8
EM-CFR	69.7	79.3	74.9	62.9
Acc-CFR	62.4	69.5	69.8	54.8
TQA				
Ans-FR	93.4	98.0	97.0	87.8
EM-FR	99.0	99.1	98.0	93.9
Acc-FR	86.4	85.2	88.4	85.8
Ans-CFR	62.1	77.9	82.2	75.6
EM-CFR	77.1	83.0	86.3	84.2
Acc-CFR	70.4	74.5	78.7	79.8
FQA				
Ans-FR	84.9	92.4	90.4	48.9
EM-FR	82.7	83.3	82.7	45.0
Acc-FR	71.1	69.2	61.6	34.5
Ans-CFR	79.2	91.2	89.6	47.8
EM-CFR	76.5	83.3	82.7	45.0
Acc-CFR	69.0	69.2	61.6	34.5
MSQ				
Ans-FR	95.3	96.4	96.2	72.6
EM-FR	99.3	100.0	99.4	80.0
Acc-FR	89.4	88.3	90.0	66.0
Ans-CFR	68.3	81.6	84.1	63.6
EM-CFR	84.0	83.1	87.3	69.6
Acc-CFR	74.5	72.3	80.0	59.0

Metric	Qwen2.5-7B	Llama-3.1-8B	Mistral-12B	Qwen3-4B
QTZ				
Ans-FR	26.1	33.0	20.8	11.1
EM-FR	23.4	31.3	21.1	10.3
Acc-FR	23.4	31.3	21.1	10.3
Ans-CFR	23.2	28.7	20.1	10.3
EM-CFR	21.3	28.5	20.7	9.8
Acc-CFR	21.3	28.5	20.7	9.8
TrQA				
Ans-FR	76.8	70.8	60.9	57.7
EM-FR	70.2	60.6	53.6	50.9
Acc-FR	57.0	48.3	45.2	41.7
Ans-CFR	50.6	60.8	55.2	50.3
EM-CFR	55.9	54.8	50.7	46.4
Acc-CFR	48.4	44.5	43.2	38.3
2Wiki				
Ans-FR	90.1	87.4	81.8	64.8
EM-FR	94.8	87.7	73.5	70.8
Acc-FR	88.2	78.3	71.5	67.2
Ans-CFR	59.6	76.5	69.1	57.4
EM-CFR	71.8	80.2	65.8	65.4
Acc-CFR	68.6	71.3	63.4	61.2
PQA				
Ans-FR	98.1	95.0	93.9	73.7
EM-FR	97.2	90.2	89.3	58.5
Acc-FR	81.3	70.2	74.0	48.8
Ans-CFR	65.2	74.8	76.5	59.0
EM-CFR	84.7	84.3	83.2	52.6
Acc-CFR	69.6	64.0	68.2	42.6

TABLE X
MEAN CONTRIBUTION-GRAPH SIZE PER QUERY.

Dataset	n	m	d_{avg}
HQA	427	660	1.55
TQA	305	964	3.16
FQA	834	188	0.23
MSQ	457	718	1.57
QTZ	73	203	2.78
TrQA	1,297	819	0.63
2Wiki	395	467	1.18
PQA	316	568	1.80

TABLE XI
FLOW VERSUS GRAPH SCORES (%), ANS-FR / ANS-CFR / EM-FR.

Dataset	Graph-Score	Single-Flow	ReFlow
HQA	88.8/49.8/84.5	89.7/51.6/86.3	**95.0/56.7/94.8**
TQA	90.1/58.3/94.0	91.2/60.5/95.3	**93.5/61.1/99.0**
FQA	66.1/59.5/56.8	75.2/68.4/66.7	**85.0/78.0/82.7**
MSQ	89.2/60.7/91.2	91.8/62.7/93.5	**95.2/66.2/99.3**
QTZ	16.5/–/15.0	21.6/–/19.5	**26.1/–/23.4**
TrQA	65.1/41.7/57.5	67.9/43.6/60.8	**76.5/51.9/74.0**
2Wiki	80.1/47.4/83.9	84.8/52.3/90.8	**90.1/57.6/94.9**
PQA	91.3/59.3/89.8	92.3/60.2/89.0	**97.9/65.4/97.2**

$|r|$; *Context* uses the 100 largest whose source is a retrieved token. The removals are computed in fp32: in bf16, the run-

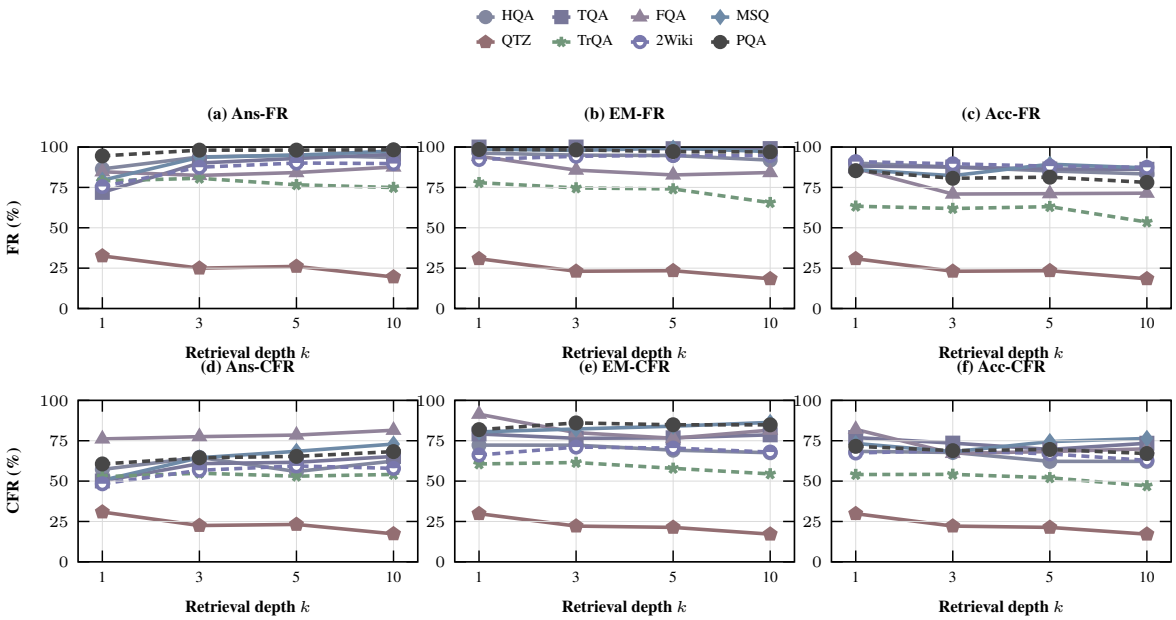


Fig. 19. Six-metric sensitivity to retrieval depth.

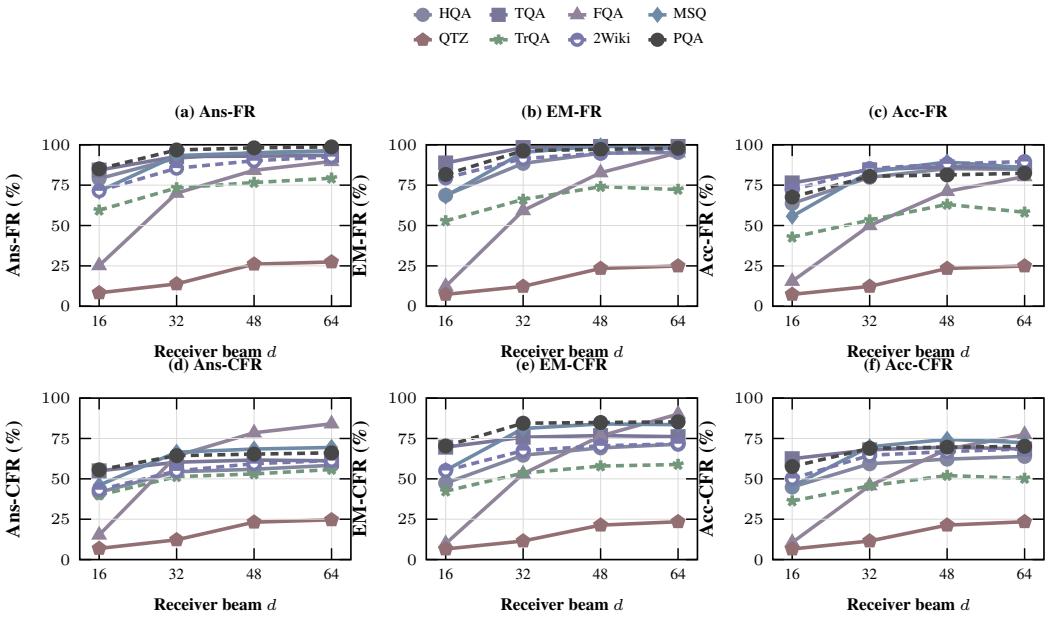


Fig. 20. Six-metric sensitivity to receiver beam.

to-run rounding of S_q ($\approx$0.1) is comparable to the effect of a single message, and with TF32 the change differs from fp32 by up to 10.

Standard faithfulness metrics. Table XIV extends Table IV with `Random`, `Attention`, and comprehensiveness by replacement, which substitutes the counterfactual alternatives for the selected tokens instead of deleting them. The model output is the geometric-mean token probability of the frozen original response.

Replacement-pool audit. Two independent LLM annotators (GPT-5.6, separate sessions, blinded and separately shuffled sheets, written guidelines) labeled 300 replacements sampled uniformly from the eight shared pools (37–38 per dataset). Each item shows the sentence with the original token and its replacement, i.e., the alternative that all methods use. Table XV reports the acceptance rate per criterion. By word class, both annotators accept 81% of numbers, 61% of proper nouns, 51% of adjectives, 17% of common nouns, and 10% of verbs; by dataset, acceptance ranges from 53% on HQA to 22% on QTZ. The main failure modes are content words that break the syntax, single-token edits inside multi-token names or numbers, and replacements that are compatible with the original (e.g., *men→women*) or near-synonyms that passed the validator. Under the weaker criterion that the replacement changes the stated fact, acceptance would be about 52–55%.

Sensitivity to the replacement pool. Table XVI regenerates

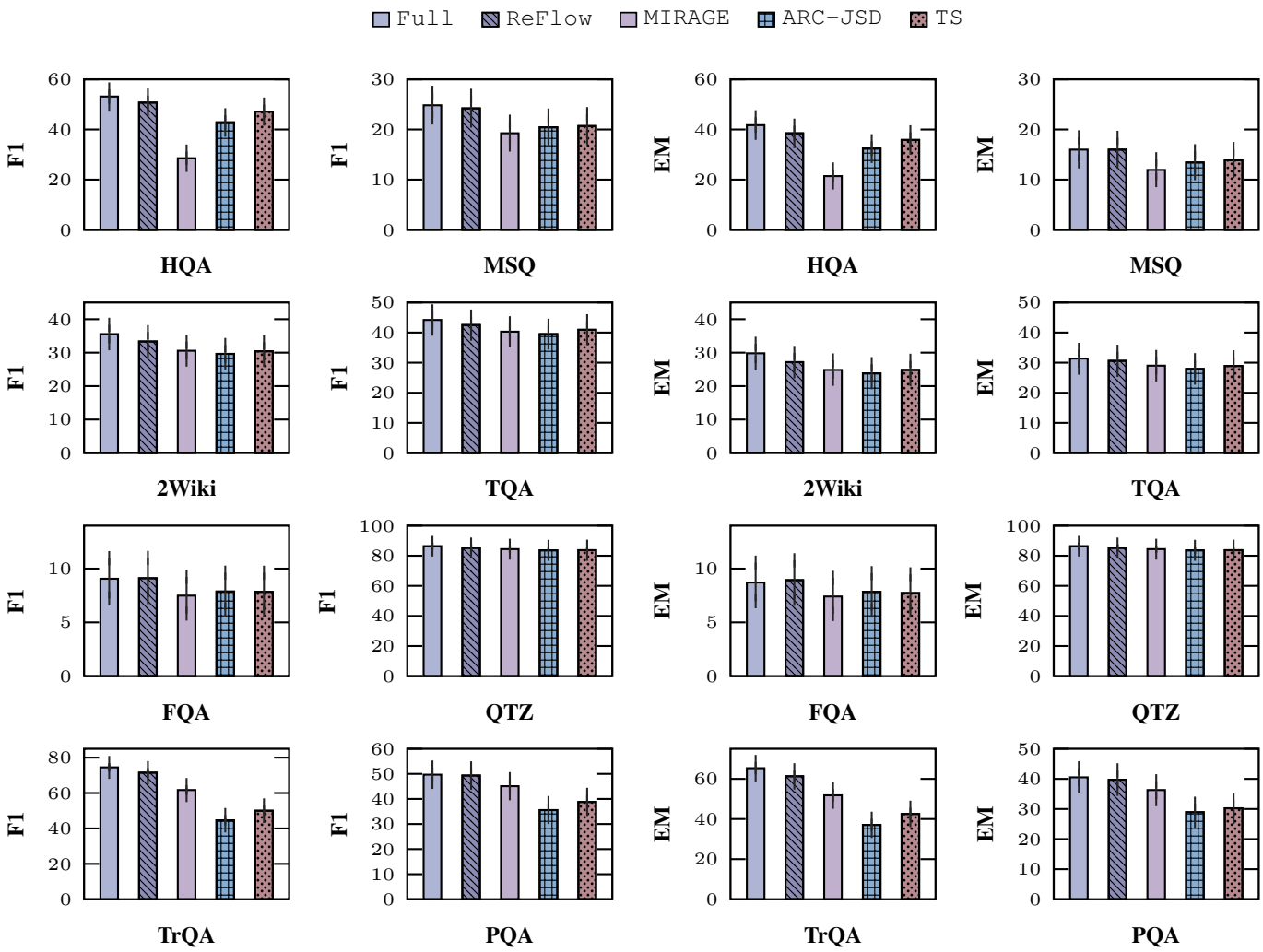
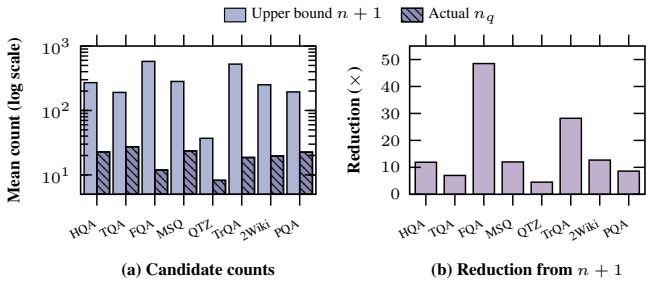


Fig. 21. RAG provenance results on all eight datasets.



(a) Candidate counts (b) Reduction from $n+1$

Fig. 22. Candidate counts vs. the theoretical bound.

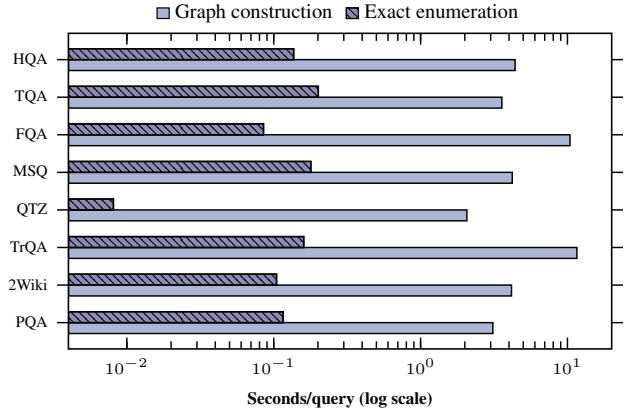


Fig. 23. Proportion of time cost of each step in ReFlow.

the pool from scratch with two other editors (same prompt, filters, and validation) and Table XVII instead picks a different valid alternative per token by changing the seed of the fixed hash. Under each new pool, ReFlow recomputes its frontier and the baselines are verified with the same pool. MiCE-RAG and RAGE are omitted from the seed comparison because their search depends on the seed-0 replacements.

Transfer to closed-source models. Table XVIII applies the token sets computed with Qwen2.5-7B-Instruct, unchanged, to two closed-source models on all valid queries of HQA, FQA, and PQA, and compares them with MiCE-RAG run with the same verification strategy. Edited contexts are answered with the same prompt as the Qwen reader; meaning-preserving controls use the same positions.

This section provides the complete proofs omitted from Section V. The proof of Lemma V.1 is already given in the

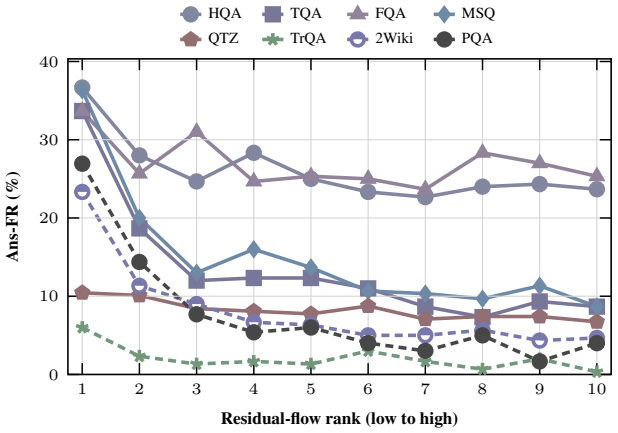


Fig. 24. Residual-flow fidelity across eight datasets.

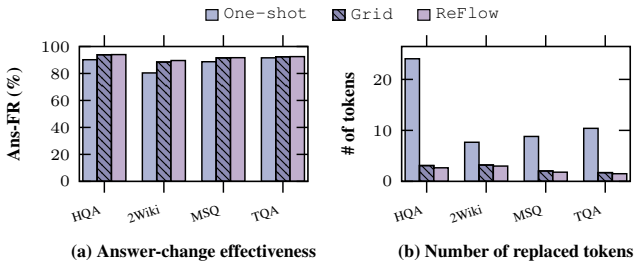


Fig. 25. Comparison of token-selection strategies.

main text and is therefore not repeated here.

G. Proof of Lemma V.2

Proof. The empty set has objective value $\Phi_q(\mathcal{G}_q)$. For every nonempty set T and every $\lambda \geq \lambda_q^{\max}$,

$$f_T(\lambda) = \lambda|T| + \Phi_q(\mathcal{G}_q \setminus T) \geq \lambda_q^{\max}.$$

The minimum-cardinality tie rule therefore selects the empty set at $\lambda_q^{\max}$ and at every larger price.

There are only finitely many token sets. Two nonidentical objective functions can be equal at only one price, while identical objectives are resolved consistently by the fixed tie-breaking rule. Thus, the returned set can change only finitely many times as the price varies.

It remains to order these intervals. For any $0 \leq \lambda_a < \lambda_b \leq \lambda_q^{\max}$, optimality at the two prices gives

$$\lambda_a|T_{\lambda_a}| + \Phi_q(\mathcal{G}_q \setminus T_{\lambda_a}) \leq \lambda_a|T_{\lambda_b}| + \Phi_q(\mathcal{G}_q \setminus T_{\lambda_b}),$$

$$\lambda_b|T_{\lambda_b}| + \Phi_q(\mathcal{G}_q \setminus T_{\lambda_b}) \leq \lambda_b|T_{\lambda_a}| + \Phi_q(\mathcal{G}_q \setminus T_{\lambda_a}).$$

Adding these inequalities and cancelling the residual-flow terms gives

$$(\lambda_b - \lambda_a)\big(|T_{\lambda_b}| - |T_{\lambda_a}|\big) \leq 0,$$

and hence $|T_{\lambda_a}| \geq |T_{\lambda_b}|$. If the two sets have the same cardinality, then

$$f_{T_{\lambda_a}}(\lambda) - f_{T_{\lambda_b}}(\lambda) = \Phi_q(\mathcal{G}_q \setminus T_{\lambda_a}) - \Phi_q(\mathcal{G}_q \setminus T_{\lambda_b}),$$

which does not depend on λ. Optimality at λ_a makes this difference nonpositive, whereas optimality at λ_b makes it nonnegative. It must therefore be zero. The two sets tie at every price, and the same tie rule returns the same set at both prices. Thus, $T_{\lambda_a} \neq T_{\lambda_b}$ implies $|T_{\lambda_a}| > |T_{\lambda_b}|$.

It remains to prove that equal endpoint solutions imply the same solution throughout the interval. Suppose $T_{\lambda_a} = T_{\lambda_b}$ but, for contradiction, some $\lambda_c \in (\lambda_a, \lambda_b)$ returns a different set T_{λ_c}. Endpoint optimality gives

$$f_{T_{\lambda_c}}(\lambda_a) - f_{T_{\lambda_a}}(\lambda_a) \geq 0 \quad \text{and} \quad f_{T_{\lambda_c}}(\lambda_b) - f_{T_{\lambda_b}}(\lambda_b) \geq 0.$$

Because the objective difference changes linearly with the price,

$$\begin{aligned} &f_{T_{\lambda_c}}(\lambda_c) - f_{T_{\lambda_a}}(\lambda_c) \\ &\quad = \frac{\lambda_b - \lambda_c}{\lambda_b - \lambda_a}\big(f_{T_{\lambda_c}}(\lambda_a) - f_{T_{\lambda_a}}(\lambda_a)\big) \\ &\qquad + \frac{\lambda_c - \lambda_a}{\lambda_b - \lambda_a}\big(f_{T_{\lambda_c}}(\lambda_b) - f_{T_{\lambda_b}}(\lambda_b)\big) \geq 0. \end{aligned}$$

On the other hand, T_{λ_c} is returned at λ_c, so the same difference is nonpositive. It must therefore be zero. Since both coefficients above are positive, both endpoint differences must also be zero. The two sets then tie throughout the interval, so the same tie rule must return the endpoint set at λ_c. This contradicts $T_{\lambda_c} \neq T_{\lambda_a}$. Therefore, $T_\lambda = T_{\lambda_a} = T_{\lambda_b}$ for every $\lambda \in [\lambda_a, \lambda_b]$.

Finally, each candidate has an integer cardinality in $\{0, 1, \ldots, n\}$, and this cardinality decreases strictly from one interval to the next. There can consequently be at most $n+1$ intervals and $|\mathcal{T}_q| \leq n+1$. □

TABLE XII
RUNNING TIME AND PEAK GPU MEMORY (QWEN2.5-7B, HQA).

Method ($k = 5$)	Time (s/query)	Peak memory (GB)
Plain inference	0.35	15.8
MIRAGE	0.59	16.5
ARC-JSD	2.39	37.2
TS	22.0	21.2
ReFlow (total)	4.68	30.3
graph construction	4.15	30.3
search (CPU)	0.14	–
verification	0.39	–
$k = 10$		
graph / plain / MIRAGE	8.14 / 0.42 / 1.08	39.0 / 16.6 / 18.5

	HQA	TQA	FQA	MSQ	QTZ	TrQA	2Wiki	PQA
Calls (mean)	2.44	1.44	3.06	1.81	6.20	5.64	2.97	2.00
Calls (max)	41	23	38	25	25	47	39	35

TABLE XIII
ACCURACY OF THE FIRST-ORDER CONTRIBUTION (100 QUERIES PER DATASET).

Dataset	Top 200 ρ	Top 200 Sign (%)	Top 200 Rel. err.	Context top 100 ρ	Context top 100 Sign (%)	Context top 100 Rel. err.
HQA	0.76	92.3	0.24	0.71	88.5	0.30
TQA	0.73	91.4	0.26	0.71	88.9	0.33
FQA	0.70	87.5	0.33	0.48	73.8	0.77
MSQ	0.74	91.6	0.25	0.70	88.5	0.29
QTZ	0.80	94.1	0.20	0.81	94.9	0.16
TrQA	0.74	91.2	0.24	0.69	86.0	0.36
2Wiki	0.73	91.7	0.24	0.68	87.9	0.30
PQA	0.76	92.1	0.22	0.74	88.9	0.29

TABLE XIV
ERASER SCORES AT THE MATCHED TOKEN BUDGET ($\times 100$): COMPREHENSIVENESS BY DELETION (DEL.) AND BY REPLACEMENT (REP.), HIGHER IS BETTER, AND SUFFICIENCY (SUF.), LOWER IS BETTER.

	HQA			TQA			FQA			MSQ			QTZ			TrQA			2Wiki			PQA		
Method	**Del.**	**Rep.**	**Suf.**	**Del.**	**Rep.**	**Suf.**	**Del.**	**Rep.**	**Suf.**	**Del.**	**Rep.**	**Suf.**	**Del.**	**Rep.**	**Suf.**	**Del.**	**Rep.**	**Suf.**	**Del.**	**Rep.**	**Suf.**	**Del.**	**Rep.**	**Suf.**
Random	0.1	0.6	34.1	1.3	1.0	34.4	1.7	1.8	29.1	0.9	0.9	44.4	2.3	3.2	3.1	-1.3	0.0	38.1	0.8	0.5	27.7	0.4	0.6	28.1
Attention	4.2	4.9	23.3	2.0	2.2	30.9	3.5	3.6	29.7	3.2	3.8	40.7	1.2	5.5	3.6	-0.5	2.2	32.1	1.3	1.4	26.9	2.4	3.1	19.3
G×I	10.8	12.4	18.6	14.4	15.5	22.6	4.6	5.2	27.0	12.5	13.6	29.5	4.0	5.1	4.2	4.3	6.9	26.3	10.9	11.9	16.7	9.2	11.4	11.7
TS	25.0	27.0	9.5	31.3	31.8	13.9	17.3	16.4	20.9	24.0	24.1	22.5	5.7	6.8	2.6	1.3	3.6	31.1	17.8	17.5	13.5	15.0	15.8	10.3
MIRAGE	9.4	11.2	21.4	27.2	29.5	15.2	8.3	10.0	26.1	21.3	23.1	21.2	4.8	5.6	3.6	6.8	10.6	21.9	18.3	19.3	11.3	16.8	20.9	7.0
ARC-JSD	9.6	11.3	13.8	29.1	32.3	14.6	17.5	19.1	23.4	28.0	30.9	18.0	3.0	6.8	3.1	5.7	9.3	30.4	19.1	20.5	10.4	16.5	19.6	9.3
ContextCite	10.7	13.0	7.7	4.7	5.0	11.3	3.6	3.9	19.7	11.8	14.3	12.7	3.9	6.4	2.5	2.6	5.5	21.5	7.9	8.1	5.4	7.6	9.7	1.5
RAGE	10.3	13.7	20.4	5.2	6.9	30.7	4.8	6.4	36.5	10.6	13.4	31.0	4.4	7.2	5.1	3.8	7.5	27.9	7.3	8.7	19.2	7.2	10.7	11.4
MiCE-RAG	10.5	12.1	19.0	13.8	15.1	22.9	3.8	4.7	28.6	12.3	13.3	29.8	4.0	5.9	4.7	4.0	6.9	26.5	10.5	11.6	17.1	8.8	11.1	11.7
ReFlow	**26.3**	**32.2**	**7.2**	**34.8**	**38.4**	**11.7**	**13.9**	**19.2**	**22.1**	**32.7**	**37.1**	**13.7**	**4.3**	**9.6**	**2.4**	**15.2**	**24.7**	**16.2**	**24.1**	**26.4**	**5.9**	**23.5**	**30.8**	**1.1**

TABLE XV
REPLACEMENT-POOL AUDIT (300 ITEMS, TWO LLM ANNOTATORS).

Criterion	**A**	**B**	**Both**	**Agree.**	κ
Grammatical	.647	.673	.613	.907	.79
Type-consistent	.620	.600	.563	.907	.80
Factually incompatible	.427	.503	.403	.877	.75
Accepted (all three)	.410	.447	.387	.917	.83

TABLE XVI
SENSITIVITY TO THE POOL EDITOR, ANS-FR / ANS-CFR (%). BEST: STRONGEST BASELINE UNDER THE SAME POOL.

Dataset	**Editor**	**ReFlow**	**Best baseline**	**MiCE-RAG**
HQA	Qwen	95.2/56.9	TS 76.9/43.5	54.8/22.1
HQA	Llama	94.6/56.5	TS 76.5/42.7	54.8/21.2
HQA	GPT	94.3/56.2	TS 77.9/43.7	55.5/22.3
MSQ	Qwen	95.5/66.2	ARC-JSD 72.2/46.8	46.2/19.4
MSQ	Llama	95.1/66.7	ARC-JSD 72.7/46.3	45.5/19.2
MSQ	GPT	95.3/66.6	ARC-JSD 74.6/48.7	47.6/21.6
FQA	Qwen	84.4/78.2	ARC-JSD 63.1/60.5	41.0/21.5
FQA	Llama	84.9/77.6	ARC-JSD 61.0/57.6	40.7/21.1
FQA	GPT	85.4/79.0	ARC-JSD 67.8/65.1	39.5/21.0

TABLE XVII
SENSITIVITY TO THE CHOSEN ALTERNATIVE (SEED OF THE FIXED HASH), ANS-FR / ANS-CFR (%). BEST: STRONGEST BASELINE UNDER THE SAME SEED.

Dataset	**Seed**	**ReFlow**	**Best baseline**
HQA	0	95.2/56.9	TS 76.9/43.5
HQA	1	94.7/56.3	TS 76.5/42.8
HQA	2	94.8/56.7	TS 76.1/42.2
MSQ	0	95.5/66.2	ARC-JSD 72.2/46.8
MSQ	1	96.0/67.0	ARC-JSD 73.1/47.2
MSQ	2	95.8/67.0	ARC-JSD 72.6/47.4
PQA	0	98.1/65.3	MIRAGE 60.0/34.2
PQA	1	98.3/65.6	MIRAGE 59.9/33.9
PQA	2	98.0/65.2	MIRAGE 59.7/34.0
FQA	0	84.4/78.2	ARC-JSD 63.1/60.5
FQA	1	84.8/78.6	ARC-JSD 63.9/61.4
FQA	2	85.1/79.0	ARC-JSD 64.4/61.8

TABLE XVIII
TRANSFER OF EXPLANATIONS TO CLOSED-SOURCE MODELS, ANS-FR / ANS-CFR (%).

Model	**Dataset**	**ReFlow**	**MiCE-RAG**
GPT-5.6-luna	HQA	59.5/30.5	39.4/14.8
	FQA	53.5/38.4	41.9/8.7
	PQA	73.5/40.0	40.3/13.5
GPT-5.6-terra	HQA	58.6/34.2	31.9/12.8
	FQA	51.7/38.7	35.1/10.9
	PQA	71.9/41.7	36.8/13.9

H. Proof of Lemma V.3

Proof. By Lemma V.2, $|T_{\lambda_a}| > |T_{\lambda_b}|$. Optimality at the two endpoints gives

$$\begin{aligned}\lambda_a\big(|T_{\lambda_a}| - |T_{\lambda_b}|\big) &\le \Phi_q(\mathcal{G}_q \setminus T_{\lambda_b}) - \Phi_q(\mathcal{G}_q \setminus T_{\lambda_a}) \\ &\le \lambda_b\big(|T_{\lambda_a}| - |T_{\lambda_b}|\big).\end{aligned}$$

Together with Equation (7), this yields $\lambda_a \le \tau \le \lambda_b$. If $\tau = \lambda_a$, then T_{λ_a} and T_{λ_b} have the same objective value at λ_a; our tie rule would select T_{λ_b} because it contains fewer tokens. Therefore, $\lambda_a < \tau \le \lambda_b$.

Suppose first that $T_\tau = T_{\lambda_b}$. By Equation (7), $f_{T_{\lambda_a}}(\tau) = f_{T_{\lambda_b}}(\tau)$, so T_{λ_a} is also optimal at τ. For any token set T and any $\lambda \in [\lambda_a, \tau]$, linearity gives

$$\begin{aligned}f_T(\lambda) - f_{T_{\lambda_a}}(\lambda) = \frac{\tau - \lambda}{\tau - \lambda_a}\big(f_T(\lambda_a) - f_{T_{\lambda_a}}(\lambda_a)\big) \\ + \frac{\lambda - \lambda_a}{\tau - \lambda_a}\big(f_T(\tau) - f_{T_{\lambda_a}}(\tau)\big) \ge 0.\end{aligned}$$

Thus, T_{λ_a} remains optimal throughout $[\lambda_a, \tau]$. Applying the same argument between τ and λ_b shows that T_{λ_b} remains optimal throughout $[\tau, \lambda_b]$. The tie rule is applied consistently at every price. Hence, no set other than T_{λ_a} and T_{λ_b} is returned in the interval.

Conversely, suppose the interval contains no other returned set. Then $T_\tau \in \{T_{\lambda_a}, T_{\lambda_b}\}$. The two endpoint sets have the same objective value at τ, and $|T_{\lambda_b}| < |T_{\lambda_a}|$, so the tie rule returns T_{λ_b}. Therefore, $T_\tau = T_{\lambda_b}$ exactly when the interval contains no additional returned set.

Finally, if $T_\tau \neq T_{\lambda_b}$, then $\tau < \lambda_b$. Moreover, T_τ cannot equal T_{λ_a} because the two endpoint sets have the same objective value at τ, while the tie rule favors the smaller set T_{λ_b}. Thus, T_τ is a new candidate. Applying Lemma V.2 on both sides of τ gives $|T_{\lambda_b}| < |T_\tau| < |T_{\lambda_a}|$, completing the proof. □

I. Proof of Theorem V.4

Proof. We prove two properties: (1) every enumerated set is an exact fixed-price solution, and (2) the recursion terminates without missing any such solution.

1) Every enumerated set is exact. The left endpoint is obtained by a minimum-cut computation at $\lambda = 0$; by Lemma V.1, it is exactly T_0. The right endpoint is the empty set, which is exactly $T_{\lambda_q^{\max}}$ by Lemma V.2. Every other recorded set is returned by a minimum-cut computation at a price τ and is therefore the exact optimizer T_τ. Hence, the algorithm introduces no spurious candidates.

2) No exact solution is missed. All distinct solutions occur in $[0, \lambda_q^{\max}]$ by Lemma V.2. Consider any call to `Divide` on

$[\lambda_a, \lambda_b]$ with exact endpoint solutions $T_{\lambda_a} \neq T_{\lambda_b}$. Lemma V.3 shows that the dividing price satisfies $\lambda_a < \tau \leq \lambda_b$. If $T_\tau = T_{\lambda_b}$, the same lemma certifies that the interval contains no other solution, so stopping is safe. Otherwise, T_τ is a new solution satisfying $|T_{\lambda_b}| < |T_\tau| < |T_{\lambda_a}|$. The two recursive calls cover $[\lambda_a, \tau]$ and $[\tau, \lambda_b]$, and both cardinality gaps are strictly smaller than $|T_{\lambda_a}| - |T_{\lambda_b}|$.

Induction on this nonnegative integer gap shows that every call terminates and enumerates every solution in its interval. Applied to the initial range $[0, \lambda_q^{\max}]$, this proves that the algorithm computes exactly all distinct T_λ. □

J. Contract-Theoretic Interpretation

Contract theory provides a natural way to understand why our search uses a token price. Its central question is how a principal can induce an agent to choose a desirable action when the action cannot simply be prescribed. The principal therefore specifies an incentive rule, and the agent chooses the action that gives the largest received payment after accounting for its action cost. The foundations of this theory were recognized by the 2016 Nobel Prize in Economic Sciences awarded to Hart and Holmström [61]. In a linear contract, the payment grows linearly with the output of an action. Such a contract is attractive because a single price makes the tradeoff between output and cost explicit and allows different actions to be compared under the same rule [70].

The same tension arises in our token-selection problem. Each token set T is a possible intervention target. Selecting more tokens may remove more contribution to the original response, but it also produces a larger intervention. Conversely, selecting very few tokens preserves a concise explanation but may leave alternative contribution paths intact. Optimizing either goal alone is therefore insufficient. Minimizing only residual flow may select an unnecessarily large set, while minimizing only the number of tokens always returns the empty set. What we need is a common mechanism for comparing the contribution removed by an intervention with the number of tokens it modifies.

Our fixed-price search follows the induced-action structure of a linear contract. Under this interpretation, the search procedure plays the role of the principal and specifies a token price λ. The fixed-price token selector plays the role of the agent and chooses T as its action. The output of this action is the contribution removed from the original response, $\Phi_q(\mathcal{G}_q) - \Phi_q(\mathcal{G}_q \setminus T)$. The action cost is $\lambda \cdot |T|$: each selected token incurs the same price λ, so the total cost grows linearly with the intervention size. The corresponding benefit-minus-cost utility is

$$u_\lambda(T) = \Phi_q(\mathcal{G}_q) - \Phi_q(\mathcal{G}_q \setminus T) - \lambda \cdot |T| = \Phi_q(\mathcal{G}_q) - f_T(\lambda). \tag{18}$$

This utility is exactly aligned with our search objective. The term $\Phi_q(\mathcal{G}_q)$ is determined by the original contribution graph and does not depend on T. Maximizing $u_\lambda(T)$ is consequently equivalent to minimizing $f_T(\lambda)$; the two objectives differ only by this constant. Thus, T_λ is precisely the action induced by the price λ, rather than an approximate solution suggested by an economic analogy.

The priced flow-network transformation gives this interpretation a concrete algorithmic meaning. Each selectable token is represented by an edge of capacity λ. Cutting this edge means paying the token price and adding the corresponding token to T. If the edge is not cut, the minimum cut must instead pay for contribution edges that continue to connect the retrieved context to the generated response. A minimum cut therefore chooses between paying for token interventions and leaving residual contribution in the graph. Its capacity is exactly $\lambda \cdot |T| + \Phi_q(\mathcal{G}_q \setminus T)$, so one minimum-cut computation returns the utility-maximizing action for the specified contract.

The price also has a direct marginal interpretation. Starting from a token set, adding another token increases the intervention cost by exactly λ. This additional token is worthwhile only when the resulting reduction in residual flow is large enough to compensate for that price. The reduction need not be an independent property of the added token: it may depend on which other tokens have already been selected because several tokens can jointly block alternative contribution paths. The contract-priced objective therefore evaluates the token set as a whole while still charging a transparent cost for its size.

As a simple illustration, suppose one action selects four tokens and removes eight units of contribution, while another selects one token and removes five units. Their utilities are respectively $8-4\lambda$ and $5-\lambda$. The four-token action is preferred when $\lambda < 1$, because the additional removed contribution justifies its three extra tokens. The one-token action is preferred when $\lambda > 1$, because intervention size has become more expensive. At $\lambda = 1$, the two actions provide the same utility, and our tie rule selects the smaller one. This example illustrates how changing the price induces different intervention targets without changing the contribution graph itself.

The two endpoints of the price range have equally clear meanings. At $\lambda = 0$, selecting tokens is free, so the induced action focuses entirely on minimizing residual flow. As the price increases, additional tokens must remove progressively more contribution to justify their cost, and the induced actions become smaller. Once $\lambda \geq \Phi_q(\mathcal{G}_q)$, every nonempty set costs at least as much as the empty-set solution before its residual flow is even counted. The empty set is therefore returned, and larger prices cannot produce a new action.

This interpretation also explains why ReFlow does not choose a single price heuristically. Before an actual counterfactual intervention, we do not know which tradeoff will yield the smallest set that changes the LLM response. A small price may identify an effective but unnecessarily large set, while a large price may identify a concise set that leaves too much contribution to the original response. We therefore vary the price and collect every token set that is optimal for some value of λ. Although the price range is continuous, Lemma V.2 proves that it induces only a linear number of distinct actions. The divide-and-conquer algorithm locates all of them exactly, and the minimum-cut transformation computes the induced action at every examined price. ReFlow then performs actual counterfactual interventions on these candidates in increasing order of size and returns the first one that changes the response. In this sense, λ is not a hyperparameter tuned on validation

data; it is an enumeration device that exposes all relevant tradeoffs between intervention size and residual contribution.

Finally, the interpretation should not be read as a behavioral assumption about the LLM. The LLM is not the agent and is not assumed to optimize an economic utility. The agent in this mapping is the fixed-price token selector, and Equation (18) introduces no new experimental metric or optimization criterion. Contract theory explains why a linear token price is a principled mechanism for balancing our two design goals. The formal correctness and efficiency of ReFlow continue to follow from the maximum-flow/minimum-cut equivalence and the candidate-enumeration results proved above.